

Лабораторна робота № 7

Вивчення ЦАП мікроконтролерів PSoC 6 фірми Cypress

Мета роботи: вивчення архітектури, принципів роботи і використання ЦАП мікроконтролерів PSoC 6; реалізація проектів в середовищі PSoC Creator 4.2 з використанням компоненти ЦАП.

Теоретичні відомості

В цьому проекті розглянемо реалізацію з допомогою компоненти DAC (12-ти розрядного) і бібліотеки периферійних драйверів CTDAC (PDL) генератор синусоїдальних сигналів за допомогою DMA.

Компонента DMA передає дані з таблиці пошуку в регістр значень ЦАП без участі процесора. Крім викликів функцій для ініціалізації та включення апаратного забезпечення, інші програмні функції не використовуються.

Схема, зображена на рис. 7.1, реалізує постійне оновлення вихідної напруги компоненти VDAC_1 за допомогою компоненти DMA_1. Передачі з DMA_1 ініціюється сигналом запуску, який формується після завершення попереднього оновлення. Компонента VDAC_1 розміщується у доступному апаратному блоці CTDAC, а сигнал вихідної напруги передається на вивід P9.6. Осцилограф, підключений до цього виводу, відображатиме форму сигналу.

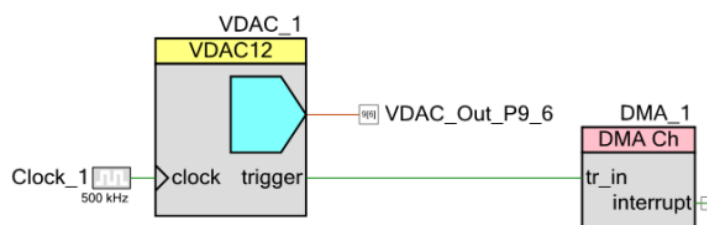


Рис. 7.1. Схема генерації синусоїдального сигналу з допомогою ЦАП

Вихід тригера VDAC_1 повідомляє компоненті DMA_1, коли VDAC_1 готовий прийняти нове значення. DMA_1 оновлює значення VDAC_1 з таблиці, яка містить наперед обчислені 100 записів коду синусоїдального сигналу в шістнадцятковій системі числення. При вхідному тактовому сигналі частотою VDAC 500 кГц частота синусоїди складає 5 кГц.

За замовчуванням у прикладі коду використовуються виклики функцій API- компонентів. Крім того, макрос PDL_CONFIGURATION можна встановити на (1u), щоб продемонструвати, як використовувати низькорівневі виклики функцій PDL CTDAC.

Основні етапи роботи з проектом:

1. Підключити стенд CY8CKIT-062 BLE до USB-порту комп'ютера.

2. Створити новий проект, реалізувати апаратну та програмну частини, запрограмувати код програми в мікроконтролер PSoC 6.

3. Підключити щуп осцилографа до виводу P9.6 на роз'ємі J2 стенду. При правильному налаштуванні осцилографа побачимо, що сигнал є синусоїдою з частотою 5 кГц та подвійною амплітудою рівною діапазону напруги 0-VDDA (рис. 10.2).

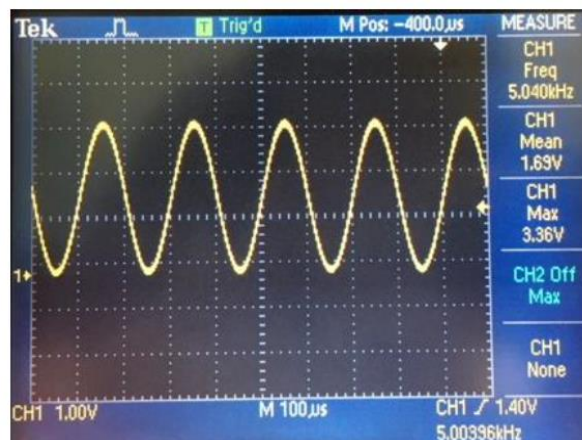


Рис. 7.2. Вигляд синусоїдального сигналу на виході VDAC

У табл. 7.2 перераховані компоненти PSoC Creator 4.2, використані в цьому проекті, апаратні ресурси, що використовуються кожною з компонент, та їх налаштування.

Табл. 7.1. Використані компоненти в середовищі PSoC Creator

Компонент	Ім'я екземпляру	Апаратні ресурси
12-ти розрядний ЦАП	VDAC_1	1 Continuous time DAC
DMA	DMA_1	1 DMA channel
Clock	Clock_1	1 Peripheral clock divider
Analog Pin	VDAC_Out_P9_6	1 Analog output pin

Компонента DMA.

Компонента DMA передає дані в пам'ять, регістри і з пам'яті та регістрів. Ці передачі відбуваються незалежно від процесора. Передача в DMA може бути налаштована довжиною байт, півслово (2 байти) або слово (4 байти). DMA починає кожну транзакцію через зовнішній тригер, який може надходити з каналу DMA (включаючи його самого), іншого каналу DMA, периферійного пристрою або центрального процесора.

Основні властивості компоненти DMA:

- Пристрій підтримує до двох апаратних блоків DMA.
- Кожний блок DMA підтримує до 16-ти каналів DMA.
- Підтримує дескриптори каналів в SRAM.
- Налаштування адрес джерела та приймача.
- Налаштування режимів передачі: одиночна передача, 1D передача (з використанням циклу по X) та 2D передача (з використанням циклів по X та Y).
- Налаштовування операції по входу тригера:

- передача одного елементу даних на тригер;
- один X (внутрішній) цикл передачі на тригер;
- весь дескриптор на тригер;
- весь канал дескрипторів на тригер.
- Налаштовування виходу тригера.
- Налаштування генерації переривань.

Основні використання DMA.

Компонента каналу DMA може використовуватися в проектах, де потрібно передавати дані без участі процесора на основі апаратного сигналу запуску від іншої компоненти.

Поширеним використанням є передача даних із пам'яті на периферійний пристрій, наприклад UART. DMA може бути викликана не повним сигналом UART FIFO. DMA буде завантажувати дані в UART до заповнення FIFO.

DMA може також використовуватися для зчитування даних з UART та зберігання їх у пам'яті. Наприклад, DMA може бути викликана не пустим сигналом FIFO. Та DMA буде передавати дані до тих пір, поки FIFO не стане пустим.

Інше поширене використання – передача даних з АЦП в пам'ять. Сигнал закінчення перетворення АЦП може використовуватися для запуску DMA для передачі даних з АЦП в пам'ять.

DMA може також використовуватися для переміщення блоків пам'яті з однієї області пам'яті в іншу (RAM в RAM, FLASH в RAM). DMA не може записувати дані в FLASH пам'ять.

Кожний канал DMA може бути запущений апаратним сигналом або записом в регістр вбудованого програмного забезпечення, або обома.

Кожний канал DMA може бути пов'язаний з дескриптором. Користувач може створити кілька дескрипторів і прив'язати їх один до одного. Користувач також може переналаштувати дескриптор, поки він не є активним дескриптором у DMA.

Підключення входів/виходів.

Розглянемо опис вхідних та вихідних з'єднань для компоненти DMA. Зірочка (*) у наведеному нижче списку вказує, що з'єднання може не відображатися на символі компоненти для умов, перелічених в описі цього вводу/виводу.

Ім'я виводу	Тип I/O	Опис
tr_in*	Digital Input	Відображається лише в тому випадку, якщо для параметра "Trigger input" встановлено значення True. Параметр вхідного запуску встановлює вхідний сигнал запуску для компоненти DMA.
tr_out*	Digital Output	Відображається лише в тому випадку, якщо для параметра "Trigger output" встановлено значення True. Параметр вихідного запуску встановлює вихідний сигнал запуску для компоненти DMA.
interrupt	Digital Output	Вихідний термінал, який дозволяє підключити компоненту переривання.

У діалоговому вікні компоненти DMA є наступні вкладки з різними параметрами.

Вкладка Channel.

Ця вкладка містить налаштування каналу DMA та кількості дескрипторів (рис. 7.3).

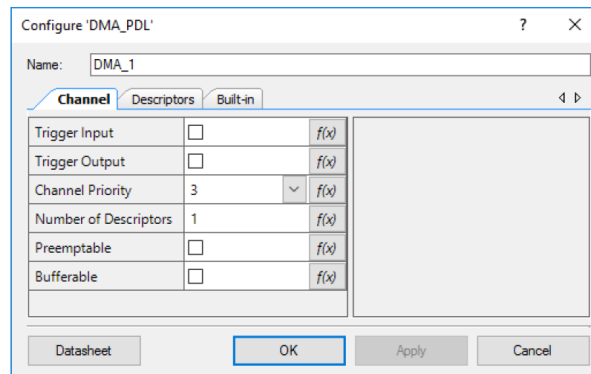


Рис. 7.3. Вигляд вкладки Channel компоненти DMA

До цієї вкладки входять наступні параметри:

- Trigger input – Вхідний параметр тригера, який встановлює вхідний сигнал тригера для компоненти DMA.

- Trigger output – Вихідний параметр тригера, який встановлює вихідний сигнал тригера для компонента DMA.

- Channel Priority – Пріоритет каналу в блоці DMA.

- Number of Descriptors – Кількість дескрипторів визначає решту частини конфігуратора. В залежності від числа, встановленого в цьому параметрі, у конфігураторі створюється стільки ж екземплярів дескрипторів.

- Preemptable – Якщо дозволено витіснення, і є канал з більш високим пріоритетом, який перебуває в очікуванні, то цей канал може витіснити поточний канал між одиничними передачами.

- Bufferable – Цей параметр вказує, чи може закінчитися транзакція запису, не дочекавшись, поки місце призначення прийме дані транзакції запису.

Вкладка, зображена на рис. 7.4, містить параметри конфігурації дескрипторів. Для кожного з дескрипторів створюється однаковий набір параметрів.

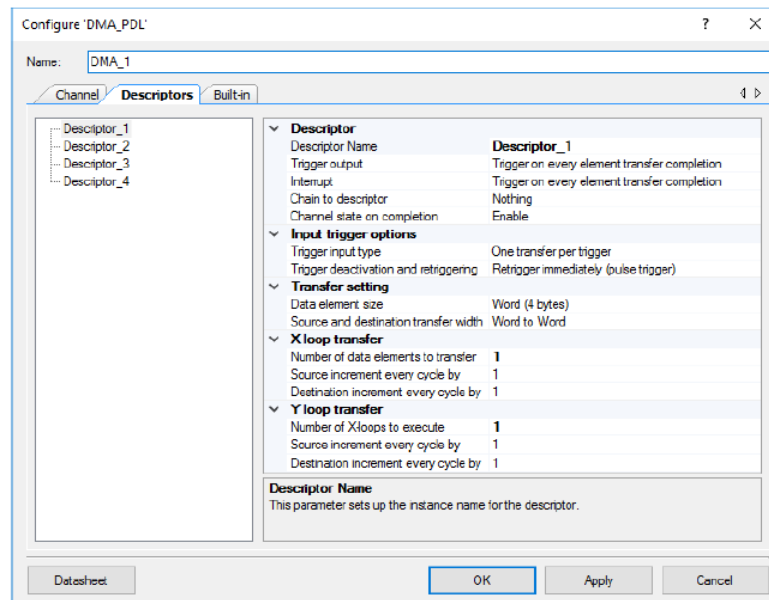


Рис. 7.4. Вигляд вкладки з параметрами конфігурації дескрипторів

Вкладка Descriptors включає функцію copy/paste налаштування властивостей дескриптора. Потрібно клікнути правою кнопкою мишки та вибрати Copy (у контекстному меню) будь-якого дескриптора (у списку дескрипторів зліва). Це копіює його налаштування у внутрішній буфер. Потім потрібно клікнути правою кнопкою мишки та вибрати Paste для будь-якого дескриптора, щоб замінити його налаштування (усі, окрім "Descriptor Name" та "Chain to descriptor"), на попередньо скопійовані з внутрішнього буфера. Також це можна виконати з допомогою клавіш Ctrl+C і Ctrl+V.

Опис параметрів дескрипторів:

- **Descriptor Name.** Цей параметр встановлює ім'я екземпляра для дескриптора. За замовчуванням використовується значення "Descriptor_N", де N – порядковий номер дескриптора.

- **Trigger output.** Цей параметр вибирає, яка подія викличе запуск тригера виходу DMA:

- запуск при кожному закінченні передачі елемента (за замовчуванням);
- запуск при кожному закінченні передачі X циклу;
- запуск при завершенні дескриптора;
- запуск при завершенні всього ланцюга дескриптора.

- **Interrupt.** Цей параметр вибирає, яка подія викличе переривання DMA:

- запуск при кожному закінченні передачі елемента (за замовчуванням);
- запуск при кожному закінченні передачі X циклу;
- запуск при завершенні дескриптора;
- запуск при завершенні всього ланцюга дескрипторів.

- **Channel state on completion** (стан каналу по завершенні). Цей параметр визначає стан (enable/disable) включення/виключення каналу, коли дескриптор завершений. У випадку "disable" канал повинен бути повторно включений (з допомогою Cy_DMA_Channel_Enable) після закінчення поточного дескриптора для подальшого функціонування. За замовчуванням цей параметр "enable".

- Chain to descriptor (коло до дескриптора). Цей параметр дозволяє налаштувати, що буде виконуватися після поточного дескриптора. Список, що відображається, показує перелік всіх дескрипторів каналу. За замовчуванням використовується значення "Nothing", яке є еквівалентне вказівнику "NULL" в коді мовою C.

Параметри тригера входу.

- Trigger input type (тип входу тригера). Тип входу тригера задасть характеристику кожного тригера:

- одна передача за запуск (за замовчуванням);
- передача одного циклу X за запуск;
- передача всього дескриптора за запуск;
- весь ланцюг дескрипторів за дескриптор.

- Trigger deactivation and retriggering (відключення і повторний запуск тригера). Цей параметр встановлює параметри деактивації тригера для дескриптора:

- негайний повторний запуск (імпульсний тригер) (за замовчуванням).
- повторний запуск після 4-х циклів Clk_Slow;
- повторний запуск після 16-ти циклів Clk_Slow;
- потрібно дочекатися повторної активації тригера.

Для опцій "Retrigger" час запуску відповідає закінченню поточної транзакції. Наприклад, вхід тригера нечутливий до рівня сигналу тригера на протязі 4/16 циклів Clk_Slow після закінчення поточної транзакції. Для повільних джерел тригера це потрібно, щоб дати сигналу достатньо часу для його зменшення, щоб не викликати небажаного спрацювання.

Transfer settings (Налаштування передачі).

- Data element size (розмір елементу даних).

Цей параметр встановлює розмір елементу даних в дескрипторі:

- байт;
- два байти (пів слова);
- чотири байти (слово) – за замовчуванням.
- Source and destination transfer width (розмір передачі джерела і приймача).

Цей параметр встановлює розміри передачі джерела і приймача:

- байт в байт;
- пів слова в пів слова;
- слово в байт;
- слово в пів слова;
- байт в слово;
- пів слова в слово;
- слово в слово (за замовчуванням).

X loop transfer (передача циклу по X)

- Number of data elements to transfer (Кількість елементів даних для передачі)

Цей параметр налаштовує кількість передач, які виконуються в циклі по X. Якщо це число рівне 1, то дескриптор визначається як одноразова передача, в

іншому випадку як 1D або 2D в залежності від параметру "Number of X-loops to execute" (Кількість циклів по X для виконання). Діапазон 1 ... 256. За замовчуванням 1.

- *Source increment every cycle by* (Джерело збільшується кожного циклу на)

Це ціле число визначає приріст адреси джерела після кожної передачі даних. Якщо значення цього параметру рівне нулю, то це рівнозначно відключенню функції приросту адреси джерела. Стосовно абсолютної адреси, одиницю приросту адреси визначається налаштуванням параметра "Source and destination transfer width" (Розмір передачі джерела і приймача). Діапазон зміни від -2048 до 2047. За замовчуванням 1.

- *Destination increment every cycle by* (приймач збільшується кожного циклу на)

Це ціле число визначає приріст адреси приймача після кожної передачі даних. Якщо значення цього параметру рівне нулю, то це рівнозначно відключенню функції приросту адреси приймача. Стосовно абсолютної адреси, одиницю приросту адреси визначається налаштуванням параметра "Source and destination transfer width" (Розмір передачі джерела і приймача). Діапазон зміни від -2048 до 2047. За замовчуванням 1.

Приріст циклу по X діє тільки всередині циклу X. Тобто кожний новий цикл по X (у випадку 2D дескриптора) починається з базової адреси srs/dst + приріст циклу по Y (накопичується під час виконання циклу Y).

В наступному прикладі показано збільшення циклів по X і Y та їх адрес:

```
cy_stc_dma_descriptor_config_t descriptor;
/* приклад екземпляру 2D дескриптора */
uint32_t * xAddress;
/* внутрішня пам'ять для зберігання збільшуваної адреси накопичувального X */
uint32_t * yAddress;
/* внутрішня пам'ять для зберігання збільшуваної адреси накопичувального Y */
uint32_t yIdx, xIdx;
/* внутрішні лічильники циклів по X та по Y */
yAddress = descriptor.srcAddress;
for(yIdx = 0; yIdx < descriptor.yCount; yIdx++)
{
    xAddress = yAddress;
    for(xIdx = 0; xIdx < descriptor.xCount; xIdx++)
    {
        (void) *xAddress; /* доступ/передача даних */
        xAddress += descriptor.srcXincrement * descriptor.srcTransferSize;
    }
    yAddress += descriptor.srcYincrement * descriptor.srcTransferSize;
}
```

Функції компоненти.

Компонента DMA_PDL використовує наступні функції. Потрібно пам'ятати, що перед кожною функцією стоїть ім'я екземпляра компоненти.

```
void DMA_1_Start (void const * srcAddress, void const * dstAddress)
```

При першому виклику (після перезавантаження системи) ця функція запускає DMA_1_Init(). При кожному виклику вона встановлює адреси джерела/ приймача в перший дескриптор (пов'язаний з цим каналом DMA), а потім активує канал з допомогою Cy_DMA_Channel_Enable().

```
void DMA_1_Init (void)
```

На основі налаштувань дескриптора ця функція запускає Cy_DMA_Descriptor_Init() для кожного дескриптора, потім ініціалізує канал та включає блок DMA з допомогою Cy_DMA_Channel_Init() та Cy_DMA_Enable() відповідно.

```
void DMA_1_Trigger (uint32_t cycles)
```

Викликає функцію драйвера PDL Cy_TrigMux_SwTrigger(). Запускає відповідний вихід тригерного мультиплексора незалежно від того, чи підключений вивід tr_in компоненти до тригерного сигналу чи ні.

Компонента також включає в себе набір специфічних для компоненти функцій-оболонок, які забезпечують спрощений доступ до базової операції прямого доступу до пам'яті. Ці функції генеруються під час побудови і всі вони мають префікс імені екземпляра компоненти.

Процедури обслуговування переривання.

Кожний екземпляр компоненти може мати переривання. Для того, щоб включити і налаштувати його, потрібно:

- Перетягнути компоненту Interrupt в TopDesign і підключити його до виводу DMA з іменем Interrupt.
- Включити переривання від компоненти з допомогою функції Cy_DMA_Channel_SetInterruptMask().
- Також додатково з допомогою функції Cy_DMA_Channel_GetStatus() в обробнику переривань можна отримати причини переривання.

Цифроаналоговий перетворювач VDAC12.

VDAC12_PDL – це 12-ти розрядний ЦАП на основі резистивної матриці. Ядро тісно пов'язано з неперервним часовим блоком (СТВ), який забезпечує просту буферизацію вихідної напруги через один з його операційних підсилювачів. Блок СТВ може додатково забезпечити буферезовану вхідну опорну напругу і функцію вибірки/зберігання для зменшення потужності при зберіганні вихідного сигналу ЦАП. Опорний сигнал на ЦАП може поступати від VDDA або іншого буферизованого сигналу через операційний підсилювач блоку СТВ. Це може бути зовнішній сигнал через GPIO або від аналогового еталону (AREF). Інтерфейс управління надає можливість управляти виходом WFG через ЦП і DMA. Це включає в ЦАП регістр управління з подвійною буферезацією, тактовий вхід для програмованої частоти оновлення, переривання по пустому буферу ЦАП для ЦП і тригер для DMA.

Основні кроки при роботі з компонентою VDAC12:

1. Перетягнути компоненту VDAC12_PDL з каталога компонент Cypress/Analog/DAC на графічний редактор середовища розробки PSoC Creator 4.2 для побудови схеми.
2. Двічі клікнути на графічному символі компоненти VDAC12_PDL. При цьому відкриється діалогове вікно налаштування параметрів.

3. Встановити потрібні параметри (джерело Vref, вихідний буфер, режим DAC і т.п.).

4. Підключити вхідні/вихідні виводи компоненти.

5. Відкрити редактор виводів для проекту і призначити вхідні і вихідні виводи мікроконтролера PSoC 6.

6. Побудувати проект, щоб перевірити правильність дизайну. Це додасть потрібні модулі в Workspace Explorer і згенерує дані конфігурації для екземпляру VDAS.

7. В файлі main.c ініціалізуємо перефінансування і запускаємо додаток.

```
/* Ініціалізація периферії CTDAC */
(void)Cy_CTDAC_Init(VDAC_CTDAC_HW, &VDAC_ctdacConfig);
#if VDAC_PARAM_ReferenceBufferUsed
/* Ініціалізація операційного підсилювача, який використовується еталонним буфером,
якщо опорний буфер був вибраний в графічному інтерфейсі. */
(void)Cy_CTB_OpampInit(VDAC_CTB_HW, VDAC_CTB_REFBUF_OPAMP_NUM, &VDAC_refBufferConfig);
#endif
#if VDAC_PARAM_OutputBufferUsed
/* Ініціалізація операційного підсилювача, який використовується вихідним буфером,
якщо вихідний буфер був вибраний в графічному інтерфейсі. */
(void)Cy_CTB_OpampInit(VDAC_CTB_HW,
VDAC_CTB_VOUTBUF_OPAMP_NUM, &VDAC_outputBufferConfig);
#endif
/* Дозвіл периферії CTDAC. */
Cy_CTDAC_Enable(VDAC_CTDAC_HW);
#if (VDAC_PARAM_ReferenceBufferUsed || VDAC_PARAM_OutputBufferUsed)
/* Дозвіл периферії CTB, якщо використовується хоча би один буфер. */
Cy_CTB_Enable(VDAC_CTB_HW);
#endif
```

8. Побудова проекту і програмування мікроконтролера PSoC 6.

Вхідні/вихідні з'єднання.

Розглянемо вхідні /вихідні з'єднання для компоненти VDAS12_PDL. Зірочка (*) після імені виводу вказує на те, що вивід може не відображатися на символі компоненти для умов, перерахованих в описі цього виводу.

- *DAC output* (Analog Output). Це вихід ЦАП. Цей вивід завжди відображається на графічному символі компоненти VDAS12_PDL.

- *trigger** (Digital Output). Цей вивід є сигналом тригера, що оновлення величини VDAS закінчено. Стан тригера є високим ("1") на протязі двох циклів PeriClk. Вивід відображається, коли встановлений прапорець Show Trigger Output Terminal і режим Update Mode не є прямим записом.

- *vref** (Analog Input). Цей вивід відобразиться тільки тоді, коли джерело опорного сигналу є зовнішнім. Це опорне джерело буде буферезоване з використанням Opamp1 CTB.

- *clock** (Digital Input). Цей вивід призначений для користувацького входу тактових імпульсів. Він відображається, коли в якості режиму Update Mode використовується буферезований запис, синхронізація по фронту імпульсів або рівню імпульсів. Максимальна тактова частота 500 кГц.

- *strobe** (Digital Input). Цей вивід призначений для користувацького входу імпульсів стробування. Він відображається, коли в якості режиму Update Mode використовується синхронізація по фронту імпульсів стробування,

негайна синхронізація по фронту імпульсів стробування або рівню імпульсів стробування.

Налаштування параметрів компонентів проекту:

На рис. 7.5, ..., рис. 7.8 зображено налаштування параметрів для компонентів, що використовуються в цьому проекті.

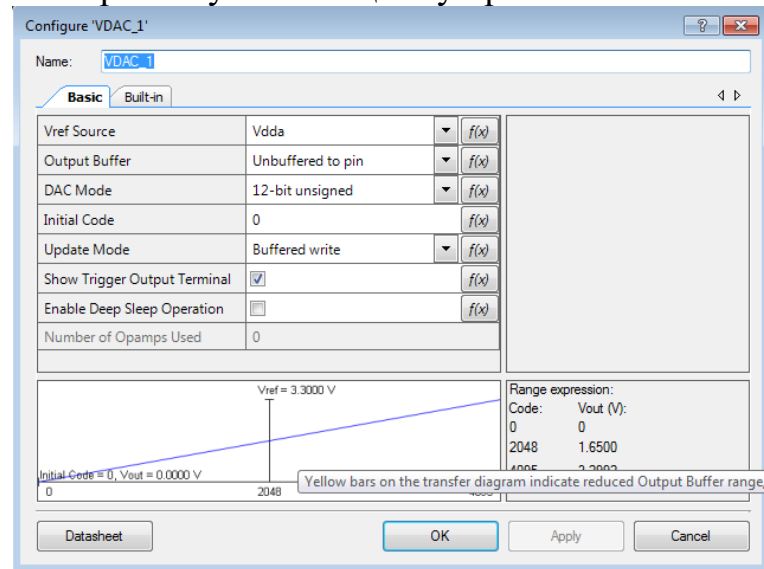


Рис. 7.5. Налаштування параметрів компоненти VDAC_1

Значення синусоїди в таблиці DMA є беззнакові. Тому режим DAC для VDAC встановлюється беззнаковий. Також потрібно включити тригерний вихідний термінал, щоб він міг бути підключений до DMA.

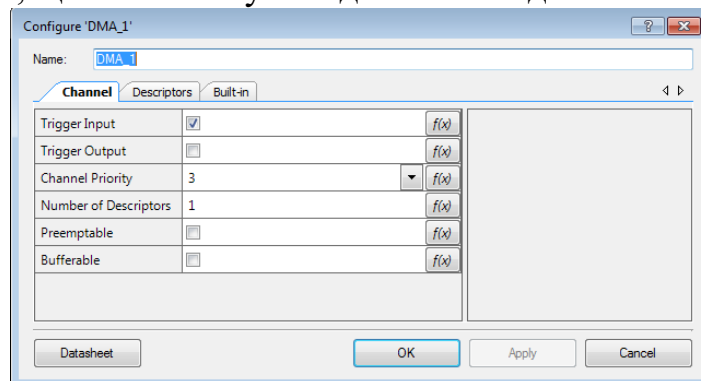


Рис. 7.6. Налаштування параметрів каналу компоненти DMA

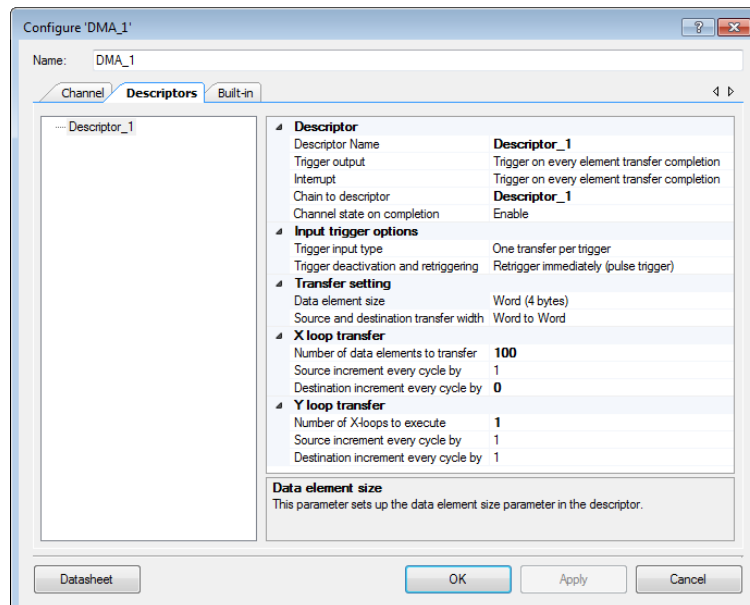


Рис. 7.7. Налаштування параметрів дескрипторів компоненти DMA

Виконати прив'язку Descriptor_1 до себе (Chain to descriptor – Descriptor_1), щоб синусоїдальний сигнал був неперервним. Кількість елементів даних для передачі потрібно встановити на 100, щоб ця кількість співпадала з розмірністю таблиці синуса. Значення адреси (адреса регістру буферезованих значень VDAC) не буде змінюватися. Тому потрібно “Destination increment every cycle by” встановити на 0.

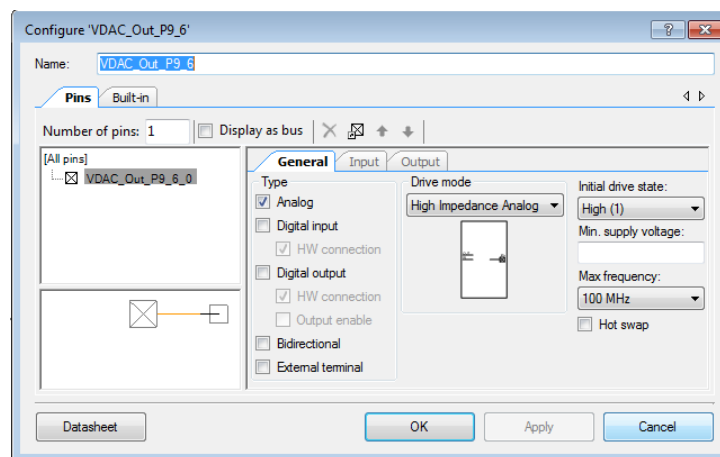


Рис. 7.8. Налаштування параметрів компоненти GPIO

Програмна реалізація проекту.

```
// LR_11_1
#include "project.h"
#include "ctdac/cy_ctdac.h"
#include "dma/cy_dma.h"

/* За замовчуванням в прикладі програми використовуються виклики функцій API
компонент. Також, встановивши макрос PDL_CONFIGURATION в значення 1, можна
використати низькорівненві виклики функції PDL CTDAC. */

#define PDL_CONFIGURATION    (0u)

/* Число точок таблиці синусоїди */
```

```

#define NUM_POINTS (100u)

/* Таблиця значень синусоїди в беззнаковому форматі. */
uint32_t sineWaveLUT[] = {0x7FF, 0x880, 0x900, 0x97F, 0x9FC, 0xA78, 0xAF1, 0xB67, 0xBD9, 0xC48,
                           0xCB2, 0xD18, 0xD79, 0xDD4, 0xE29, 0xE77, 0xEC0, 0xF01, 0xF3C, 0xF6F,
                           0xF9A, 0xFBE, 0xFDA, 0xFEE, 0xFFA, 0xFFF, 0xFFA, 0xFEE, 0xFDA, 0xFBE,
                           0xF9A, 0xF6F, 0xF3C, 0xF01, 0xEC0, 0xE77, 0xE29, 0xDD4, 0xD79, 0xD18,
                           0xCB2, 0xC48, 0xBD9, 0xB67, 0xAF1, 0xA78, 0x9FC, 0x97F, 0x900, 0x880,
                           0x7FF, 0x77E, 0x6FE, 0x67F, 0x602, 0x586, 0x50D, 0x497, 0x425, 0x3B6,
                           0x34C, 0x2E6, 0x285, 0x22A, 0x1D5, 0x187, 0x13E, 0x0FD, 0x0C2, 0x08F,
                           0x064, 0x040, 0x024, 0x010, 0x004, 0x000, 0x004, 0x010, 0x024, 0x040,
                           0x064, 0x08F, 0x0C2, 0x0FD, 0x13E, 0x187, 0x1D5, 0x22A, 0x285, 0x2E6,
                           0x34C, 0x3B6, 0x425, 0x497, 0x50D, 0x586, 0x602, 0x67F, 0x6FE, 0x77E};

/* В головній функції main виконуються наступні дії:
1. Ініціалізуються компоненти VDAC та DMA
2. DMA постійно оновлює значення VDAC з таблиці синусів */

int main(void)
{
    /* Дозвіл глобальних переривань. */
    __enable_irq();

#ifdef PDL_CONFIGURATION
    /* Налаштування VDAC з джерелом опорної напруги VDDA
    * та небуферезованим виходом.
    * Функція FastInit налаштує VDAC для без знакового режиму і буферезованих
    * оновлень з тактовою частотою 500 кГц за замовчуванням. */
    Cy_CTDAC_FastInit(CTDAC0, &Cy_CTDAC_Fast_VddaRef_UnbufferedOut);

    /* Налаштування Trigger Mux для маршрутизації вихідного тригера VDAC
    * з вхідним тригером DMA */
    (void)Cy_TrigMux_Connect(TRIG14_IN_PASS_TR_CTDAC_EMPTY,
        TRIG14_OUT_TR_GROUP0_INPUT50, CY_TR_MUX_TR_INV_DISABLE,
        TRIGGER_TYPE_PASS_TR_CTDAC_EMPTY);
    (void)Cy_TrigMux_Connect(TRIG1_IN_TR_GROUP14_OUTPUT7,
        TRIG1_OUT_CPUSS_DW1_TR_IN0, CY_TR_MUX_TR_INV_DISABLE,
        TRIGGER_TYPE_TR_GROUP_OUTPUT__LEVEL);

    /* Дозвіл апаратного блоку CTDAC */
    Cy_CTDAC_Enable(CTDAC0);

    DMA_1_Start(sineWaveLUT, &CTDAC0->CTDAC_VAL_NXT);
#else
    VDAC_1_Start();
    DMA_1_Start(sineWaveLUT, (uint32_t *)&(CTDAC0->CTDAC_VAL_NXT));
#endif

    for(;;)
    {
    }
}

```

Завдання:

1. Реалізувати на виході GPIO P9.6 мікроконтролера PSoC 6 з допомогою ЦАП гармонічний сигнал змінної частоти згідно алгоритму: перші 2-а періоди сигнал з частотою 100 Гц, наступні 4 періоди – сигнал з частотою 200 Гц і потім – 8 періодів сигнал з частотою 400 Гц. З допомогою АЦП виміряти цей сигнал і виводити на термінал, використовуючи UART.